SEO PACKAGE

1. SEO Title

SQL DISTINCT Explained: How to Remove Duplicate Rows and Count Unique Values Correctly

2. SEO Subtitle

Learn what a duplicate row really is, why DISTINCT judges the whole row, and when it's secretly hiding a broken join.

3. Featured Quote

> "DISTINCT should clean data that is genuinely repeated — not silence a query that is quietly broken."

4. Meta Description

Master SQL DISTINCT: remove duplicate rows, count unique values with COUNT(DISTINCT), and learn when DISTINCT hides a join bug. A clear, beginner-friendly guide.

5. URL Slug

`sql-distinct-remove-duplicate-rows`

6. Keywords

SQL DISTINCT, remove duplicate rows SQL, COUNT DISTINCT, SQL unique values, DISTINCT vs GROUP BY, SQL duplicate rows, database duplicates, SQL SELECT DISTINCT, count unique rows SQL, SQL query optimization, SQL for beginners, SQL interview questions, deduplicate SQL, multiple columns DISTINCT, SQL join duplicates, learn SQL, SQL tutorial, database fundamentals, SQL aggregate functions, SQL performance

7. Tags

SQL, DISTINCT, GROUP BY, Databases, SQL Tutorial, Coding Interview, Data Analysis, Backend Development, Software Engineering, SQL Basics, Query Optimization, Learn SQL, Data Structures

---

8. Introduction

Your data is often lying to you about how much you actually have. A report that shows five customer rows might really represent just two countries wearing disguises — and if you count the raw rows, you'll hand your boss a number that is simply wrong.

This is one of the most common early mistakes in SQL, and it shows up everywhere: analytics dashboards, admin panels, billing exports, and especially coding interviews. Interviewers love questions about duplicate rows because they reveal whether you actually understand how a database *thinks*, or whether you're just memorizing keywords. Do you know what a "row" is? Do you know what "unique" means when more than one column is involved? Do you know the difference between counting rows and counting distinct values?

`DISTINCT` is the keyword that rips off the mask and shows you each real thing exactly once. It looks like the easy button — but it hides two traps that quietly break real queries. By the end of this guide you'll know exactly when to reach for `DISTINCT`, how to count unique values correctly, how it differs from `GROUP BY`, and — most importantly — when `DISTINCT` is secretly covering up a bug instead of fixing one.

This is a fundamental skill for anyone learning SQL, working with data structures at the database level, or preparing for a technical interview in software engineering.

9. Problem Overview

Here's the problem in plain terms.

A **row** is a single line of data in a table — one record, like one entry on a guest list. A **duplicate row** is a row whose values repeat one or more other rows. Picture a guest list where the name *Priya* got written down three times. Those two extra Priyas are duplicates: the same answer showing up more than once.

Duplicates cause three practical headaches:

1. **Wrong counts.** If you count rows to answer "how many *different* things do we have?", duplicates inflate your number.
2. **Noisy output.** A list meant to show unique options is buried under repeats.
3. **Hidden bugs.** Sometimes duplicates aren't real data at all — they're the symptom of a join that matched too many rows.

The task: given a table that may contain repeated values, return each unique value **once**, and be able to count how many unique values exist. Sounds trivial. The trap is in the details of *what counts as unique*.

10. Example

Imagine a `customers` table:

id	name	city	country
1	Priya	Mumbai	India
2	Arjun	Delhi	India
3	Meera	Pune	India
4	John	London	UK
5	Sarah	Leeds	UK

Ask a simple question: **"How many countries do we sell to?"**

If you run:

```
SELECT country FROM customers;
```

You get **five rows** — `India, India, India, UK, UK`. Staring at that raw list, you might report "five." But there are really only **two** countries hiding in there. The repeats bury the real answer.

The correct answer to "how many countries" is **2**. Everything below is about getting to that 2 reliably — and understanding what happens when you ask the question slightly differently.

11. Intuition

An experienced engineer doesn't jump to a keyword. They first ask: *what does "the same" mean here?*

Two rows are duplicates only if **every value you're looking at** matches. That single idea unlocks everything. If you're only looking at `country`, then `India` and `India` are the same. But the moment you also look at `city`, the rows `Mumbai, India` and `Delhi, India` become *different* — because as **pairs**, they don't match, even though `India` repeats.

So the mental model is: **collapse identical things down to one copy.** Take the triple Priya and squash it into a single Priya. Take five country rows and, if you only care about country, squash them into two.

Once you hold that picture — *"keep one of each unique combination, drop the rest"* — the SQL almost writes itself. The keyword `DISTINCT` is just the database doing that collapsing for you. The skill isn't memorizing the word; it's knowing **which columns to point it at**, because that decides what "unique" means.

12. Brute Force Solution

Idea: Before you know about `DISTINCT`, how would you remove duplicates by hand? You'd pull every row, then walk through them one by one, keeping a running list of values you've already seen. If a value is new, keep it. If you've seen it before, skip it.

In pseudocode over the fetched rows:

```
seen = set()
unique = []
for row in all_rows:
    if row not in seen:
        seen.add(row)
        unique.append(row)
```

Algorithm: 1. Fetch all rows from the database into your application. 2. Loop through each row. 3. Track seen values in a set. 4. Emit a row only the first time you see it.

Advantages: - Easy to understand; no SQL knowledge beyond `SELECT`. - Full control in application code.

Disadvantages: - You transfer **every** row over the network, including all the duplicates — wasteful. - The database is far better optimized for this than your app loop. - More code, more places for bugs, harder to maintain.

Time complexity: O(n) to scan the rows, where `n` is the number of rows. **Space complexity:** O(k) for the `seen` set, where `k` is the number of unique values.

It works, but it's the wrong layer. The database can do this for you — often faster and without shipping the noise across the wire.

13. Optimal Solution

Drop the word `DISTINCT` right after `SELECT`, and the database does the sweeping for you:

```
SELECT DISTINCT country FROM customers;
```

Five noisy rows become two clean ones:

country
India
UK

That's the whole spell — one keyword.

The multi-column rule. When you list more than one column, `DISTINCT` looks at the **combination**, not each column alone:

```
SELECT DISTINCT city, country FROM customers;
```

city	country
Mumbai	India
Delhi	India
Pune	India
London	UK
Leeds	UK

Notice `India` appears three times here — and that's correct. As **pairs**, `(Mumbai, India)` and `(Delhi, India)` are different. `DISTINCT` dedupes the whole combo you selected, not one column.

Counting unique values. To answer "how many different countries?", put `DISTINCT` *inside* `COUNT`:

```
SELECT COUNT(DISTINCT country) FROM customers;   -- returns 2
```

Compare that with the plain version:

```
SELECT COUNT(country) FROM customers;            -- returns 5
```

Query	Meaning	Result
`COUNT(country)`	How many rows have a country?	5
`COUNT(DISTINCT country)`	How many *different* countries?	2

`COUNT` answers *how many total*. `COUNT(DISTINCT ...)` answers *how many different*. That single difference is the one that saves you from reporting the wrong number.

14. Python Code

Here's how you'd use these queries from clean, production-quality Python with the standard `sqlite3` module:

```
import sqlite3


def count_unique_countries(db_path: str) -> int:
    """Return the number of distinct countries in the customers table."""
    with sqlite3.connect(db_path) as conn:
        cursor = conn.execute(
            "SELECT COUNT(DISTINCT country) FROM customers"
        )
        return cursor.fetchone()[0]


def list_unique_countries(db_path: str) -> list[str]:
    """Return each country exactly once, in sorted order."""
    with sqlite3.connect(db_path) as conn:
        cursor = conn.execute(
            "SELECT DISTINCT country FROM customers ORDER BY country"
        )
        return [row[0] for row in cursor.fetchall()]
```

Let the database deduplicate; let Python just carry the clean result.

15. Code Walkthrough

- **`with sqlite3.connect(db_path) as conn:`** — Opens the database and guarantees the connection is committed and closed even if an error occurs. The `with` block is the lazy, correct way to manage resources.
- **`conn.execute("SELECT COUNT(DISTINCT country) ...")`** — Pushes the deduplication *into the database*. Only the final number crosses back into Python — not five rows, not a million.
- **`return cursor.fetchone()[0]`** — `COUNT` returns a single row with a single column, so `fetchone()[0]` grabs the scalar directly.
- **`SELECT DISTINCT country ... ORDER BY country`** — Returns each country once. `ORDER BY` makes the output stable and predictable, which matters for tests and readable reports.
- **`[row[0] for row in cursor.fetchall()]`** — Unpacks each single-column row into a flat list of strings.

The key architectural decision: deduplication happens in SQL, not in a Python loop. Less data moves, less code exists, fewer bugs hide.

16. Dry Run

Let's trace `SELECT COUNT(DISTINCT country) FROM customers` against our table.

Rows scanned, in order, with a running set of seen countries:

Step	Row country	Seen set before	New?	Seen set after
1	India	{}	yes	{India}
2	India	{India}	no	{India}
3	India	{India}	no	{India}
4	UK	{India}	yes	{India, UK}
5	UK	{India, UK}	no	{India, UK}

Final distinct set: `{India, UK}` → size **2**. The query returns `2`.

Now trace `SELECT COUNT(country)` — no `DISTINCT`. It never checks the set; it just increments a counter for every non-null row: 1, 2, 3, 4, 5 → **5**. Same table, different question, different answer. That gap is the whole lesson.

17. Complexity Analysis

To remove duplicates, the database must compare values, and there are two common strategies under the hood:

- **Sorting:** Order all the rows so identical copies sit next to each other, then keep the first of each run. This costs roughly **O(n log n)** time.
- **Hashing:** Drop each value into labeled buckets (a hash table) and ignore any value already present. This is closer to **O(n)** time but uses **O(k)** extra memory for the buckets, where `k` is the number of unique values.

Space: O(k) for the set of distinct values (plus temporary sort space if sorting is used).

Why does this matter? On a few thousand rows you'll never feel it. On a hundred million rows, that sorting or hashing is **real work** — CPU, memory, and sometimes disk. So use `DISTINCT` with intent, not by reflex. It is not free; it's just cheap enough to be invisible on small tables.

18. Alternative Solutions

The natural alternative is `GROUP BY`:

```
SELECT country FROM customers GROUP BY country;
```

For a plain unique list, `DISTINCT` and `GROUP BY` return the **same answer**. So which do you use?

You want…	Use	Why
Just the unique values	`DISTINCT`	Less typing, clearer intent, reads cleaner
Uniques **plus a number per group**	`GROUP BY`	Only `GROUP BY` can also aggregate

`GROUP BY`'s superpower is that it can *crunch numbers per group*:

```
SELECT country, COUNT(*) AS customer_count
FROM customers
GROUP BY country;
```

country	customer_count
India	3
UK	2

Freeze this frame — it settles the question forever. Same rows in, two exits. Asking *"what different things exist?"* → `DISTINCT`. Asking *"how many, or how much, for each thing?"* → `GROUP BY`. That number-per-group only comes from `GROUP BY`.

19. Edge Cases

- **Empty table:** `SELECT DISTINCT country` returns zero rows; `COUNT(DISTINCT country)` returns `0`. Safe, no error.
- **All rows identical:** `DISTINCT` collapses them to a single row. `COUNT(DISTINCT ...)` returns `1`.
- **All rows unique:** `DISTINCT` changes nothing; the output equals the input — and you paid the deduplication cost for no benefit.
- **NULL values:** `DISTINCT` treats all `NULL`s as *one* value and keeps a single `NULL`. But `COUNT(DISTINCT country)` **ignores NULLs entirely** — so a column of `India, NULL, UK` gives `COUNT(DISTINCT country) = 2`, not 3. This trips people constantly.
- **Extra column sneaks in:** Add one more column and your "unique" list can suddenly explode, because uniqueness now spans that column too.

20. Common Mistakes

1. **Counting raw rows instead of distinct values.** Using `COUNT(country)` when you meant `COUNT(DISTINCT country)` — reporting 5 countries when there are 2.
2. **Adding a helpful extra column.** "I wanted a clean list of countries, so I wrote `SELECT DISTINCT country`, then added `city` to be helpful — and suddenly India was back three times." `DISTINCT` **always spans every column you select.** Only ask for the columns you actually want deduped.
3. **Thinking `DISTINCT` applies to one column.** `SELECT DISTINCT a, b` dedupes the **pair**, not just `a`. It is not `DISTINCT(a), b`.
4. **Using `DISTINCT` to hide a broken join.** Duplicates are often screaming that a join matched too many rows. Slapping `DISTINCT` on top just paints over the bug. Ask *why* the duplicates appeared first.
5. **Reaching for `DISTINCT` by reflex on huge tables.** On a hundred million rows, the sort or hash is real, measurable work. Don't pay it unless you need it.
6. **Forgetting NULL handling.** `COUNT(DISTINCT col)` skips `NULL`, which can quietly under-count.

21. Interview Questions

Expect follow-ups like these:

- What exactly makes two rows duplicates? *(Every selected value matches.)*
- What's the difference between `COUNT(*)`, `COUNT(col)`, and `COUNT(DISTINCT col)`?
- Does `SELECT DISTINCT a, b` dedupe on `a`, on `b`, or on the pair?
- When would `DISTINCT` and `GROUP BY` give the same result, and when would they differ?
- How does the database *implement* `DISTINCT` under the hood, and what's the performance cost?
- You see duplicate rows after a `JOIN`. Would you add `DISTINCT`? Why or why not?
- How does `DISTINCT` handle `NULL` values?

22. Similar Problems

- **LeetCode 196 — Delete Duplicate Emails.** Directly about identifying and removing duplicate rows; same "what makes a row unique" reasoning.
- **LeetCode 182 — Duplicate Emails.** Find values that appear more than once — the inverse of `DISTINCT`, and a natural `GROUP BY ... HAVING COUNT(*) > 1`.
- **LeetCode 1050 — Actors and Directors Who Cooperated At Least Three Times.** Reinforces `GROUP BY` with aggregation, the exact tool `DISTINCT` *can't* replace.

- **LeetCode 595 — Big Countries.** Practices filtering and selecting the right columns, where over-selecting changes your result set.

All four sharpen the same instinct: knowing precisely which columns define "unique," and whether you need a list or a count-per-group.

23. Key Takeaways

- A **duplicate row** is the same combination of values appearing more than once.
- `DISTINCT` keeps one of each unique combination and sweeps the repeats.
- It judges the **whole set of selected columns together** — add a column, and "unique" changes.
- `COUNT(DISTINCT col)` answers *how many different*; plain `COUNT` answers *how many total*.
- `DISTINCT` for uniques; `GROUP BY` when you also need a number per group.
- `DISTINCT` costs real work on big tables — use it with intent, not reflex.
- Never let `DISTINCT` become the tape over a leaky join. Fix the duplicate at its source.

24. Watch the Video

Seeing the rows collapse in real time makes this click faster than any paragraph can. Watch the full walkthrough here — **{LINK}** — where we sweep five noisy rows down to two clean ones, live, and settle the `DISTINCT` vs `GROUP BY` question with a single frame worth screenshotting.

25. About the Series

This is **SQL Lesson 9** from **Fun with Learning Technology** — a series that teaches real backend and database skills in short, focused lessons built on intuition, not memorization. Each lesson takes one concept most tutorials rush past, slows it down, and shows you the trap *before* you fall into it. Follow along from the start to build a genuinely solid SQL foundation, one clear idea at a time.

26. Call To Action

If this cleared up duplicates for you, **like the video on YouTube** — it genuinely helps the channel reach more developers. **Subscribe** so you don't miss the next lesson, and **subscribe to the Substack** for the written deep-dives like this one. Got a question or a case where `DISTINCT` burned you? **Drop a comment** — I read them. And if a teammate keeps counting raw rows, **share this** with them.

SOCIAL MEDIA

LinkedIn Post

Your data is often lying about how much you actually have.

A query that returns five customer rows might really represent just two countries. Count the raw rows and you'll report the wrong number — to a dashboard, a stakeholder, or an interviewer.

The fix is one SQL keyword: DISTINCT. But it hides two traps worth knowing before you rely on it.

Trap 1: DISTINCT judges the WHOLE row, not one column. `SELECT DISTINCT country` gives you clean, unique countries. Add `city` "to be helpful" and duplicates come flooding back — because uniqueness now spans both columns. DISTINCT always dedupes every column you select, together. Only ask for the columns you actually want deduped.

Trap 2: DISTINCT is often a band-aid over a broken join. Duplicate rows are frequently a symptom of a join that matched too broadly. Slapping DISTINCT on top silences the symptom and hides the bug. Before reaching for it, ask *why* the duplicates appeared.

A few things worth remembering: • COUNT(col) answers "how many total." COUNT(DISTINCT col) answers "how many different." Those are not the same question. • Want just unique values? DISTINCT. Want a number per group (count, sum)? GROUP BY. • On a hundred million rows, DISTINCT does real work (sorting or hashing). Use it with intent, not reflex.

Small keyword, big consequences. Getting it right is the difference between a correct report and a confident wrong one.

What's a case where DISTINCT quietly burned you? I'd love to hear it.

SQL #Databases #SoftwareEngineering #CodingInterview #LearnSQL

Twitter/X Thread

1/ Your SQL data is lying to you.

Five rows might really be two things wearing disguises. Count the raw rows and you'll report the wrong number.

One keyword fixes it — but it hides two traps. 🪤

2/ First, what's a duplicate?

A row is one line of data. A duplicate is the same combination of values showing up more than once.

Think of a guest list where "Priya" got written 3 times. Two of those are duplicates.

3/ Enter DISTINCT.

```
SELECT DISTINCT country FROM customers
```

It keeps one of each unique value and sweeps the repeats. Five noisy rows → two clean ones. One keyword.

4/ Trap #1: DISTINCT looks at the WHOLE row you selected.

```
SELECT DISTINCT city, country
```

Mumbai/India and Delhi/India BOTH survive — as pairs they're different, even though India repeats. DISTINCT dedupes the combo, not one column.

5/ This bit me early:

I wanted unique countries, wrote `DISTINCT country`, then added `city` to be helpful.

Suddenly India was back 3 times. Adding a column changed what "unique" means. Only select the columns you actually want deduped.

6/ Counting? Two different questions:

`COUNT(country)` → 5 (how many rows) `COUNT(DISTINCT country)` → 2 (how many different)

Reporting 5 when the answer is 2 is how wrong numbers reach your boss.

7/ Trap #2 (the big one): DISTINCT is overused as a band-aid.

Duplicate rows are often screaming that a JOIN is too broad. DISTINCT on top just paints over the bug.

Ask WHY the duplicates showed up before you sweep them.

8/ DISTINCT vs GROUP BY?

For a plain unique list → same answer, DISTINCT reads cleaner.

But GROUP BY can also crunch numbers per group (count, sum per country). DISTINCT can't.

9/ The rule that settles it forever:

"What different things exist?" → DISTINCT "How many / how much for each thing?" → GROUP BY

Screenshot that.

10/ And it's not free. Under the hood DISTINCT sorts or hashes every row. Invisible on 1,000 rows, real work on 100 million.

Use it with intent — not as tape over a leaky join.

Full lesson 👉 {LINK}

Facebook Post

Ever run a SQL query, get five rows back, and confidently report "five"... when the real answer was two?

That's the duplicate-row trap, and almost everyone hits it early. In this lesson I break down DISTINCT — the one keyword that shows you each real value exactly once — in plain, beginner-friendly language.

You'll learn what a duplicate row actually is, why DISTINCT looks at the whole row (not just one column), how to count unique values correctly with COUNT(DISTINCT), and the sneaky case where DISTINCT is quietly hiding a broken join instead of fixing it. We also finally settle DISTINCT vs GROUP BY with a rule you can screenshot.

No jargon, no fluff — just the intuition that makes it stick. Watch it here 👉 {LINK}

Reddit Summary

What DISTINCT actually does (and two ways it quietly bites you)

Sharing a breakdown I put together on `DISTINCT`, since I see the same confusion come up a lot.

The core idea: a duplicate row is the same *combination of selected values* repeated. `DISTINCT` keeps one of each and drops the rest.

Two things that trip people up:

1. **DISTINCT spans every column you select — not one.** `SELECT DISTINCT city, country` dedupes on the *pair*, so `(Mumbai, India)` and `(Delhi, India)` both survive. If you add a column to a `DISTINCT` query and duplicates "come back," that's why. It's not a bug.

2. **DISTINCT is often a band-aid over a bad join.** If a query returns duplicates you didn't expect, the honest move is to ask why — usually a join matched too many rows. `DISTINCT` will hide that symptom while leaving the actual bug in place.

Also worth internalizing: - `COUNT(col)` = how many rows. `COUNT(DISTINCT col)` = how many different values. `COUNT(DISTINCT ...)` also ignores NULLs. - DISTINCT and GROUP BY give the same result for a plain unique list; GROUP BY is the one that can also aggregate per group. - It's not free — it sorts or hashes under the hood. Fine on small tables, real cost on huge ones.

Curious what edge cases others have hit with it — NULL handling and join-induced duplicates are the two that got me.

GITHUB README

```
# SQL DISTINCT — Removing Duplicate Rows & Counting Unique Values

> SQL Lesson 9 · Fun with Learning Technology

## Problem

Given a table that may contain repeated values, return each unique value
exactly once, and count how many unique values exist. Raw row counts lie:
five rows can represent just two real values.

Example `customers` table:

| id | name  | city   | country |
|----|-------|--------|---------|
| 1  | Priya | Mumbai | India   |
| 2  | Arjun | Delhi  | India   |
| 3  | Meera | Pune   | India   |
| 4  | John  | London | UK      |
| 5  | Sarah | Leeds  | UK      |

Question: "How many countries do we sell to?" → **2**, not 5.

## Intuition

Two rows are duplicates only if **every selected value** matches. If you
select just `country`, the three India rows collapse to one. Add `city` and
they're distinct again, because uniqueness now spans the pair. The whole skill
is choosing which columns define "unique."
```

```
## Approach

- `SELECT DISTINCT country` → each country once.
- `SELECT DISTINCT city, country` → dedupes the **combination**, not one column.
- `COUNT(DISTINCT country)` → how many *different* values (2).
- `COUNT(country)` → how many rows (5).
- Use `GROUP BY` instead when you also need a number per group (count, sum).
- DISTINCT deduplicates via sorting or hashing under the hood — real cost on
  large tables. Don't use it to mask a join that matched too many rows.

## SQL

```sql
-- unique values
SELECT DISTINCT country FROM customers;

-- count unique values (ignores NULLs)
SELECT COUNT(DISTINCT country) FROM customers;

-- uniques PLUS a number per group
SELECT country, COUNT(*) AS customer_count
FROM customers
GROUP BY country;
```

Python Solution

```
import sqlite3


def count_unique_countries(db_path: str) -> int:
    """Return the number of distinct countries in the customers table."""
    with sqlite3.connect(db_path) as conn:
        cursor = conn.execute("SELECT COUNT(DISTINCT country) FROM customers")
        return cursor.fetchone()[0]


def list_unique_countries(db_path: str) -> list[str]:
    """Return each country exactly once, sorted."""
    with sqlite3.connect(db_path) as conn:
        cursor = conn.execute(
            "SELECT DISTINCT country FROM customers ORDER BY country"
        )
        return [row[0] for row in cursor.fetchall()]
```

Complexity
```

Strategy	Time	Space
Sorting	O(n log n)	O(n) temp
Hashing	O(n)	O(k) buckets

`n` = number of rows, `k` = number of unique values. Negligible on small tables; real work on very large ones.

Gotchas

- DISTINCT spans **every** selected column, not one.
- `COUNT(DISTINCT col)` ignores `NULL`.
- Unexpected duplicates often mean a broken/broad JOIN — fix the cause, don't mask it with DISTINCT.

Video

📺 Watch the full walkthrough: {LINK}

Article

📖 Full written deep-dive: {LINK} ```

Quick Review

SQL tell: query returns repeated rows hiding true variety. What do you reach for?

DISTINCT. When repeated entries obscure how many unique entities exist, add DISTINCT to collapse identical rows into one.

What granularity does DISTINCT compare — a column or the row?

The entire selected row. Uniqueness is judged across all columns in the SELECT list combined, not per individual column.

Time complexity of DISTINCT deduplication?

O(n log n) if sorted, or ~O(n) with hashing, over n rows. The engine must sort or hash to detect duplicates.

Space complexity of DISTINCT?

O(k) for k distinct rows kept (hash set), up to O(n) worst case, plus sort buffers if using a sort-based plan.

Common bug: SELECT DISTINCT col1, col2 — what do people wrongly expect?

They think each column is deduped separately. It isn't — DISTINCT dedupes the whole combined row, so unique pairs still produce repeats.

Follow-up: DISTINCT vs GROUP BY for getting unique values?

Both dedupe. GROUP BY is needed when aggregating (COUNT, SUM) per group; DISTINCT is simpler when you only want unique rows, no aggregates.

---

sql Lesson 9: DISTINCT — Removing Duplicates (The DISTINCT clause is a foundational SQL operator used to filter out redundant rows from result sets to ensure data uniqueness. By instructing the database engine to perform a sorting or hashing operation on requested columns, it allows analysts to generate precise lists of distinct entities. You should reach for this tool whenever your query returns repetitive entries that obscure the underlying count or variety of your dataset.) — free Python cheat sheet